

Project Installation Guide

ChimpRec — Master thesis project

November 5, 2025

Overview

This document explains how to install and run the project on macOS and Linux. The repository mainly contains Jupyter notebooks. The steps below show how to install Python 3.12 (macOS with Homebrew), create and activate a virtual environment, install dependencies, and launch the notebooks.

1 Prerequisites

- Git (to clone the repository)
- Homebrew (macOS) or appropriate package manager for Linux
- A C compiler and build tools may be needed for some Python packages (Linux)

2 macOS (with Homebrew) — Install Python 3.12

The recommended way to install Python 3.12 on macOS is Homebrew. The commands below cover both Apple Silicon and Intel macs.

1. Install Homebrew (if not already installed)

Open Terminal and run:

```
# macOS: install Homebrew (if not installed)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the on-screen instructions from the Homebrew installer.

2. Install Python 3.12 via Homebrew

```
# Install Python 3.12
brew update
brew install python@3.12
```

Homebrew installs the formula in a location that may not be on your default PATH. To make the python3.12 binary available, add the following to your shell profile.

3. Add Python 3.12 to PATH

For Apple Silicon (M1/M2) Homebrew default:

```
# Add to ~/.zshrc or ~/.bash_profile (Apple Silicon)
echo 'export PATH="/opt/homebrew/opt/python@3.12/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For Intel macs (Homebrew default location):

```
# Add to ~/.zshrc or ~/.bash_profile (Intel)
echo 'export PATH="/usr/local/opt/python@3.12/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

4. Verify installation

```
python3.12 --version
pip3.12 --version
```

You should see Python 3.12.x.

3 Linux

Two common approaches on Linux are using your distribution's packages (or PPAs) and using pyenv. Below are instructions for Ubuntu (via deadsnakes) and for Fedora, plus a pyenv option that is cross-distro.

3.1 Ubuntu (20.04 / 22.04) via deadsnakes PPA

```
# Add deadsnakes PPA
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update

# Install Python 3.12 and venv-dev packages
sudo apt install -y python3.12 python3.12-venv python3.12-dev python3.12-distutils

# Optional: ensure pip is available
curl -sS https://bootstrap.pypa.io/get-pip.py | sudo python3.12
```

3.2 Fedora (example)

```
# Fedora may already provide newer Python; use dnf or build from source
sudo dnf install -y python3.12 python3.12-devel
```

3.3 pyenv (cross-distro, recommended when distro packages are not available)

```
# Install build dependencies (Ubuntu example)
sudo apt update
sudo apt install -y make build-essential libssl-dev zlib1g-dev \
```

```
libbz2-dev libreadline-dev libsqlite3-dev wget curl llvm \
libncursesw5-dev xz-utils tk-dev libxml2-dev libxmlsec1-dev libffi-dev liblzma-dev
```

```
# Install pyenv (follow instructions at https://github.com/pyenv/pyenv)
curl https://pyenv.run | bash
```

```
# Add pyenv init to shell (example for ~/.bashrc or ~/.zshrc)
echo 'export PATH="$HOME/.pyenv/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
echo 'eval "$(pyenv virtualenv-init -)"' >> ~/.bashrc
source ~/.bashrc
```

```
# Install Python 3.12 with pyenv
pyenv install 3.12.0
pyenv global 3.12.0
```

4 Clone the repository

```
git clone https://github.com/diegRock/ChimpRec.git
cd ChimpRec
```

5 Create and activate a virtual environment

Using the installed Python 3.12 binary:

```
# Create virtual environment (cross-platform)
python3.12 -m venv .venv
```

```
# Activate (macOS/Linux)
source .venv/bin/activate
```

```
# Upgrade pip and tooling
pip install --upgrade pip setuptools wheel
```

6 Install project dependencies

If the repository provides a requirements.txt:

```
python3.12 -m venv venv
source ./venv/bin/activate
pip install -r Code/requirements_linux_macos.txt
```

If there is no requirements file, and the project mainly uses Jupyter notebooks, you might run:

```
pip install jupyterlab notebook numpy pandas matplotlib seaborn scikit-learn
```

Adjust packages to match the repository requirements.

7 Run the notebooks and selecting the correct kernel in VSCode

You can run notebooks in a browser via jupyter notebook / jupyter lab or open them inside VSCode. If you use VSCode, follow the steps below to ensure the notebook runs with the virtual environment (Python 3.12) you created.

1. Install Jupyter support in the environment

Activate your virtual environment and install ipykernel and any notebook packages:

```
source .venv/bin/activate
pip install ipykernel jupyterlab notebook
```

Register the virtual environment as a Jupyter kernel (this makes it selectable in VSCode):

```
# from the activated venv
python -m ipykernel install --user --name=chimprec-venv --display-name "Python 3.12 (chimprec)"
```

2. Install VSCode extensions

In VSCode install the following extensions (if not already installed):

- Python (Microsoft)
- Jupyter (Microsoft)

3. Open the project in VSCode

In VSCode: File → Open Folder... → select the Chimprec folder.

4. Select the correct interpreter (optional but recommended)

Open the Command Palette (Ctrl/Cmd+Shift+P) and run:

```
Python: Select Interpreter
```

Choose the interpreter that points to your project's virtual environment (for example, .venv/bin/python). Selecting the interpreter helps the Python extension locate installed packages and tools.

5. Open a notebook and choose the kernel

Open any .ipynb file in the repository. In the top-right corner of the notebook editor you will see the kernel picker (it shows the current kernel/interpreter). Click it and choose the kernel named "Python 3.12 (chimprec)" (or the display name you set when installing the ipykernel).

If the kernel is not listed:

- Ensure the venv is activated and ipykernel was installed in that venv.
- Restart VSCode (Developer: Reload Window).
- From the Command Palette run: "Notebook: Select Notebook Kernel" and choose the installed kernel.
- As a fallback, run the notebook server externally (jupyter lab) and connect VSCode to that server (in the kernel picker choose "Select Another Kernel..." → "Existing Jupyter Server").

6. Run cells

With the correct kernel selected, run notebook cells as usual (Shift+Enter or use the play buttons). The selected kernel ensures code executes using Python 3.12 and the packages installed in your project's virtual environment.

8 Notes & Troubleshooting

- If Homebrew installs `python@3.12` but `python3.12` is not found, ensure the Homebrew opt path is added to your shell PATH (see macOS section).
- On Linux, some Python packages require native libraries (e.g., `libxml2`, `libatlas`, `BLAS/LAPACK`). Install the appropriate `-dev` packages for your distro.
- If you get errors building a wheel, install build tools (`gcc`, `make`) and dev headers for the dependency in question.
- Use a dedicated virtual environment to avoid version conflicts with the system Python.
- If VSCode shows a kernel but running cells raises `ImportError` for packages you installed, double-check that `ipykernel` was created from the same Python interpreter where you installed those packages.

If you want a repo-specific README

If you provide the repository's `readmeinstallationcontent(pasteithere)`, I will rewrite/adapt this LaTeX file to reflect

Good luck — enjoy exploring the notebooks!